

CODE SPORT



Sandya Mannarswamy

In this month's column, we continue our discussion of binary instrumentation tools, and look at a few popular dynamic instrumentation frameworks.

Last month, we had discussed various memory errors such as uninitialised memory reads and bit undefined errors. We also discussed the concept of shadow memory, wherein extra meta-data maintained in memory is used to track the state of each memory location in an application's address space. Shadow memory contains information on the state of each memory location used by the application, so that it can be used by an analysis tool to detect various errors. For instance, consider that the analysis tool supports a shadow memory infrastructure wherein every word of the application's heap memory has a corresponding shadow word memory; and whenever the application's heap memory is initialised during the application's execution, the corresponding shadow word records this information. So if we are asked to find out whether a particular memory location is initialised or not, at the point of its use, all that the tool needs to do is to look up the corresponding shadow word which contains that information.

In this month's column, we continue our discussion of shadow memory. A couple of our readers, R Manohar and N Sudeep Kumar, had questions on how dynamic instrumentation is typically implemented, and how they can write their own custom tools on top of a dynamic binary instrumentation framework. We will address these questions in this column. First, let's take a quick look at the various approaches to application instrumentation.

Instrumentation approaches

While our discussion (last month) on memory error checking was limited to binary instrumentation, there are different instrumentation approaches possible, depending on the purpose for which the instrumentation is being done. For example, if users want to find out how many branch instructions are encountered during program execution, the simplest approach is to do a binary instrumentation that looks for branch op-codes in the execution of instructions. On the other hand,

consider a user who wants to find out how many times a function 'foo' was called with the first argument being zero and the second argument being 1, from the parent 'bar', when bar was called with the first argument being *NULL*. Such an arbitrary instrumentation requires deeper application-path analysis, and is typically done easily on source code or an intermediate representation, where the code representation is quite close to high-level language constructs. The various popular instrumentation approaches are enumerated below.

Source-code instrumentation

This is typically done at the pre-processor level, wherein the instrumentation code in the same high-level language is inserted into the application source code; and the application is then compiled and linked normally. Since the instrumentation code is part of the application code as it compiles, it can be optimised by the compiler. The instrumentation code kicks in and executes when the application binary is executed at runtime.

Compile-time instrumentation

This is during the compilation of the source file, and the instrumentation is typically performed by the compiler on the intermediate representation. This facilitates the use of the compiler's knowledge of the application in optimising the placement of the instrumentation code, and allows it to improve the performance of the instrumented executable. For example, the GCC compiler allows injection of *gprof* profile code through compiler-based instrumentation. GCC also supports an option known as *-finstrument-functions*, which automatically traces all function entry and exit points for user code functions.

Link-time instrumentation

This instrumentation happens during the post-linking phase of the object files, and is also known as object-code